Universidad Simón Bolívar

Departamento de Computación y Tecnología de la Información

CI-2693 - Laboratorio de Algoritmos 3

Trimestre Septiembre Diciembre 2026

Informe de Proyecto
Proyecto 1: Análisis de una red de transporte urbano

Estudiantes:

David Garrido – 2310610

Kevin Gomes – 2310170

Noviembre de 2026

1. Diseño

1.1. Arquitectura de RedTransporte y Composición

Para reutilizar las implementaciones del Laboratorio 1 sin modificar su lógica original ni incurrir en colisiones de nombres por herencia múltiple, se diseñó la clase **RedTransporte** aplicando el patrón de composición. Esta clase encapsula una instancia de **GrafoMatriz** y una de **GrafoLista**, coordinando la carga de datos y la ejecución de las consultas.

Dado que los archivos identifican las estaciones con cadenas de texto pero los grafos operan con índices enteros de 0 a n - 1 según su orden de aparición, **RedTransporte** centraliza el mapeo mediante un unordered_map (**unordered_map<string, int> nombre_a_id**) que traduce nombres a índices en O(1) promedio, y un arreglo (**vector<string> id_a_nombre**) que traduce índices a nombres en O(1) estricto. Adicionalmente, mantiene la matriz siguiente de tamaño **n x n** para reconstruir caminos mínimos y los vectores **grado_in** y **grado_out** de tamaño n para registrar los grados de cada estación durante la lectura.

1.2. Costo de las Operaciones Utilizadas en el Proyecto

La siguiente tabla resume el costo de las operaciones de RedTransporte y de los métodos de GrafoMatriz y GrafoLista efectivamente utilizados en el proyecto, para una red de n estaciones, m tramos y grado de salida d+(u):

Operación en el Proyecto	Matriz (GrafoMatriz)	Lista (GrafoLista)	RedTransporte (Auxiliares)
Mapear estación (nombre <-> id**)**	No aplica	No aplica	O(1) promedio en unordered_map y O(1) en vector.
Inicializar estructura (setGrafo**)**	$O(n^2)$	O(1) (constructor vacío)	O(n^2) para matriz siguiente y O(n) para vectores de grado.
Registrar estación (agregarVertice**)**	O(n) (redimensiona filas)	O(1) amortizado	O(n) total por estación al insertar en ambas

			estructuras.
Registrar tramo (agregarArista**)**	O(1)	O(1) amortizado	O(1) (actualiza siguiente[u][v], grado_in[v] y grado_out[u]).
Obtener referencia (obtener...Referencia**)**	O(1)	O(1)	Retorna puntero/referencia a M o adj sin copiar memoria.
Consultar / actualizar peso de (u, v)	O(1)	O(d+(u))	En matriz es acceso directo M[u][v]; en lista requiere búsqueda lineal.
Recorrer vecinos de u	O(n)	O(d+(u))	En matriz recorre n columnas; en lista itera solo sobre adj[u].

1.3. Justificación del Uso de Estructuras por Consulta

Uso de GrafoMatriz para Floyd-Warshall, RUTA, ZONAS, DIAMETRO y CERRAR

Se seleccionó GrafoMatriz para estas consultas porque todas dependen de consultar y relajar tiempos mínimos entre pares arbitrarios (i, j). El acceso directo O(1) a cualquier celda M[i][j] permite que cada paso del triple ciclo de Floyd-Warshall se ejecute en tiempo constante, logrando una complejidad óptima de $O(n^3)$. Asimismo, permite verificar el alcance mutuo en ZONAS (dist[i][j] != INF y dist[j][i] != INF) y buscar el máximo tiempo en DIAMETRO en $O(n^2)$. Si se usara GrafoLista para Floyd-Warshall, consultar el peso (i, j) costaría O(d+(i)) y registrar nuevos caminos transitivos requeriría insertar arcos dinámicamente; al densificarse el grafo, d+(i) se aproxima a n, lo que degradaría Floyd-Warshall de $O(n^3)$ a $O(n^4)$.

Uso de GrafoLista para SALTOS y ALCANZABLES

Se utilizó GrafoLista para los recorridos BFS y DFS porque su operación central consiste en explorar únicamente los arcos salientes de cada estación visitada. Iterar

sobre adj[u] cuesta O(d+(u)), lo que permite recorrer toda la componente alcanzable en tiempo O(n + m). Además, como los tramos se insertan con push_back durante la lectura, GrafoLista conserva automáticamente el orden de aparición del archivo exigido como desempate en SALTOS. Si se ejecutaran estos recorridos sobre GrafoMatriz, descubrir los vecinos de u obligaría a inspeccionar las n columnas de su fila en O(n), degradando la complejidad de O(n + m) a O(n^2) y perdiendo el orden cronológico en que fueron leídos los tramos.

Optimización mediante Referencias Directas y Preprocesamiento de Grados

Se añadieron los métodos **obtenerMatrizReferencia()** y **obtenerListaReferencia()** para operar directamente sobre la memoria de **M** y **adj** desde RedTransporte. Si en el BFS de SALTOS se hubiera utilizado el método original **lista.vecinos(u)**, el programa habría tenido que reservar un nuevo vector y copiar por valor los vecinos en cada nodo extraído de la cola; usar una referencia constante (**const vector<...>&**) evita todas las copias dinámicas manteniendo el acceso en O(1).

2. Algoritmos

2.1. Algoritmo de Floyd-Warshall y Consulta RUTA

Funcionamiento

Al inicializar **RedTransporte**, la matriz **M** se configura en **INF** con 0 en la diagonal principal (**M[i][i] = 0**) y siguiente en -1. Por cada tramo leído u -> v de peso t, se asigna **M[u][v] = t** y **siguiente[u][v] = v**. Luego, **ejecutarFloydWarshall**() ejecuta tres ciclos anidados k, i, j en orden creciente de 0 a n - 1. Para evitar desbordamientos al sumar **INF**, se comprueba que **dist[i][k] != INF** y **dist[k][j] != INF**, además de emplear enteros de 64 bits (long long).

Reconstrucción de rutas

En **consultaRuta(origen, destino)**, se obtienen los índices **u** y **v** en O(1). Si **dist[u][v] == INF** se imprime **NO HAY RUTA**, y si **u == v** se imprime el origen con 0 min. En el caso general, un índice **actual = u** avanza iterativamente mediante **actual = siguiente[actual][v]** mientras **actual != v**, imprimiendo el nombre de cada estación y el tiempo final **dist[u][v]**.

Complejidad temporal y espacial

El preprocesamiento de **Floyd-Warshall** toma tiempo $O(n^3)$ por los tres ciclos anidados de tamaño n y espacio $O(n^2)$ para almacenar las matrices **M** y **siguiente**. Cada consulta **RUTA** se responde en tiempo O(L), donde L <= n es la cantidad de estaciones de la ruta óptima.

2.2. Búsqueda en Amplitud (BFS) y Consulta SALTOS

Funcionamiento y exploración de vecinos

El método **consultaSaltos(origen, destino)** busca la ruta con menor número de tramos usando BFS sobre la referencia constante de **GrafoLista**. Si **u == v**, imprime 0 tramos. De lo contrario, inicializa los vectores **visitado(n, false)**, **pred(n, -1)**, **saltos(n, 0)** y una cola **Q** donde inserta **u**. Al extraer el nodo **actual** de la cola, se itera sobre **adj[actual]**; por cada vecino no visitado se marca como visitado, se registra **pred[vecino] = actual**, se calcula **saltos[vecino] = saltos[actual] + 1** y se encola. Si **vecino == v**, el ciclo termina de inmediato, asegurando la ruta más corta en aristas según el orden de aparición en el archivo.

Reconstrucción del camino

Si **v** no fue alcanzado, se reporta **NO HAY RUTA**. Si se alcanzó, se reconstruye la ruta retrocediendo desde **v** hasta **u** mediante el arreglo **pred** y guardando los índices en un vector camino. Finalmente, se imprime camino en orden inverso indicando (1 tramo) o (k tramos) según corresponda.

Complejidad temporal y espacial

La complejidad temporal es O(n + m), ya que inicializar los vectores toma O(n), cada vértice se encola a lo sumo una vez (O(n)) y su lista de adyacencia se recorre una sola vez sumando O(m) pasos en total. La complejidad espacial auxiliar es O(n) por la cola **Q** y los vectores **visitado**, **pred**, **saltos** y **camino**.

2.3. Consulta RESUMEN y Conteo de Zonas

Funcionamiento

El método **consultaResumen**() recorre linealmente los vectores **grado_out** y **grado_in** de **0** a **n - 1** actualizando el máximo solo con desigualdad estricta (>), lo

que preserva la estación de menor índice en caso de empate. Para reportar el total de zonas, llama al método auxiliar **calcularCantidadZonas**().

Complejidad temporal y espacial

2.4. Búsqueda en Profundidad / Amplitud y Consulta ALCANZABLES

Funcionamiento e implementación

La consulta **consultaAlcanzables(origen)** determina el conjunto de estaciones a las que se puede llegar desde una estación de origen dada. Para ello, realiza un recorrido en amplitud (BFS) sobre la estructura GrafoLista. Se inicializa una cola **q** y un vector booleano visitado de tamaño **n**. Al visitar un nodo, se insertan sus vecinos no visitados en **q** y se agregan a una estructura **set<string>** alcanzables.

Ordenamiento y presentación de resultados

El uso de **set<string>** inserta automáticamente los nombres de las estaciones en orden alfabético. Al finalizar el recorrido BFS, se imprime la cantidad de elementos contenidos en el conjunto seguido del carácter **:** y, posteriormente, la lista ordenada de estaciones separadas por espacio.

Complejidad temporal y espacial

Como recorrer el grafo completo con cada una de sus aristas cuesta O(n + m), y se debe agregar cada estación alcanzable con un costo de O(log A) para cada nodo alcanzable, obtenemos O(A log A), por lo tanto, **consultaAlcanzables(origen)** tiene una complejidad temporal de O(n + m + A log A) y complejidad espacial O(n).

2.5. Componentes Fuertemente Conexas y Consulta ZONAS

Funcionamiento y agrupación

La consulta **consultaZonas()** calcula e imprime las componentes fuertemente conexas (Zonas) de la red. Para determinar si dos estaciones **i** y **j** pertenecen a la misma zona, el algoritmo consulta la matriz resultante del algoritmo de Floyd-Warshall y verifica si se cumple la condición de alcance mutuo: **arista(i,j) ≠ INF** y **arista(j,i) ≠ INF**. Se utiliza un vector booleano de marcaje para garantizar que cada estación sea asignada a una única zona

Criterios de ordenamiento

Para garantizar el formato de salida y la determinación de las zonas, cada zona individual se representa internamente como un **set<string>**, manteniendo las estaciones internas ordenadas alfabéticamente. Además, todas las zonas se agrupan en una estructura **set<set<string>>**, la cual ordena dinámicamente las zonas tomando como clave de comparación su primera estación alfabética.

Complejidad temporal y espacial

Al recorrer cada arista y su opuesta, se recorre un conjunto de tamaño **n x n** obteniendo una complejidad temporal $O(n^2)$ y complejidad espacial O(n) para los vectores que almacenan las zonas.

2.6. Trayecto Mínimo Más Largo y Consulta DIAMETRO

Funcionamiento y manejo de desempates

La consulta **consultaDiametro()** busca el par de estaciones **(u, v)** con **u ≠ v** que maximice la distancia más corta entre todas las conexiones posibles de la red. Se efectúa un recorrido exhaustivo de tamaño **n x n** sobre la matriz de distancias mínimas. En caso de empate en el tiempo máximo alcanzado, el algoritmo prioriza el par que posea el menor índice en la estación de origen **u**, y en caso de persistir el empate, el menor índice en la estación de destino **v**.

Complejidad temporal y espacial

Al recorrer un conjunto de tamaño **n x n** la complejidad temporal es $O(n^2)$ y complejidad espacial O(1).

2.7. Simulación de Fallos en la Red y Consulta CERRAR

Funcionamiento y recálculo sobre la red modificada

La consulta **consultaCerrar(estacion)** simula el retiro de una estación dada. Para realizar este cálculo de forma aislada sin alterar el estado global de la red, se genera una copia temporal **dist_nuevo** basada en **matriz_original** (que contiene únicamente los tiempos directos de los tramos). En esta copia se inhabilitan todos

los arcos entrantes y salientes fijándolos en **INF** (con **dist_nuevo(id,id) = 0**). Luego, se ejecuta nuevamente el algoritmo de Floyd-Warshall sobre la matriz temporal omitiendo la estación cerrada como nodo intermediario **k**.

Conteo de pares desconectados (p) y perjudicados (q)

Se comparan la matriz de caminos previa **dist_viejo** y la matriz recalculada **dist_nuevo** para todo par de estaciones **(i, j)** donde **i ≠ id, j ≠ id y j ≠ i**:

- Contador **p**: Se incrementa si existía un camino (**dist_viejo(i,j) ≠ INF**) y tras el cierre deja de ser alcanzable (**dist_nuevo(i,j) = INF**).
- Contador **q**: Se incrementa si el par sigue conectado (**dist_nuevo(i,j) ≠ INF**), pero su nuevo tiempo mínimo es estrictamente mayor que el original (**dist_nuevo(i,j) > dist_viejo(i,j)**).

Complejidad temporal y espacial

Su complejidad temporal es de $O(n^3)$ debido al algoritmo de Floyd-Warshall y la complejidad espacial es de $O(n^2)$ requerido para instanciar la matriz temporal.

3. Casos de Prueba Propios

3.1. Caso de Prueba 1

Para este caso (**red1.txt** y **consultas1.txt**), se construyó una red que contiene 6 estaciones y 8 tramos con el propósito de evaluar el comportamiento de las operaciones básicas a menor escala. Esta prueba permite verificar la ejecución correcta de las métricas generales, la reconstrucción de rutas secuenciales simples, el cálculo de alcanzables mediante BFS y la simulación de cierres aislados, garantizando además el manejo adecuado de errores ante estaciones inexistentes sin interrumpir el flujo del programa.

3.2. Caso de Prueba 2

Esta prueba (**red2.txt** y **consultas2.txt**) se enfocó en evaluar el criterio de desempate en la consulta RESUMEN dentro de una red de 7 estaciones y 10 tramos. Se verificó que al existir múltiples estaciones con el mismo grado máximo de salida (Petare, Los_Dos_Caminos, Chacao y Chacaito), se seleccione e imprima la de menor índice registrado (Chacaito), aplicando exactamente la misma lógica para el grado máximo de entrada al desempatar entre Chacaito y Miranda a favor de Miranda.

3.3. Caso de Prueba 3

En este último caso (**red3.txt** y **consultas3.txt**) se diseñó una red de mayor complejidad estructural con 12 estaciones y 22 tramos para garantizar la estabilidad del sistema en redes amplias y altamente interconectadas. La prueba permitió validar la búsqueda de caminos mínimos de larga distancia, casos borde con saltos nulos sobre una misma estación, y la consistencia en el recálculo de rutas y pares desconectados o perjudicados tras la ejecución repetida del comando CERRAR.

4. Estado del Proyecto

4.1. Cumplimiento de Requerimientos Funcionales y No Funcionales

El programa cumple con todos los requerimientos especificados en el enunciado. La ejecución por consola (./p1 <red.txt> <consultas.txt>) construye simultáneamente la lista y la matriz de adyacencia sin emplear bibliotecas externas de grafos, ejecuta **Floyd-Warshall** una única vez al inicio y reutiliza la matriz resultante en **RESUMEN**, **RUTA**, **ZONAS** y **DIAMETRO**, recalculándola únicamente sobre una copia local en **CERRAR**. Todas las consultas respetan el formato exacto de salida y los criterios de desempate verificados con el caso de prueba oficial.

4.2. Manejo de Errores y Estado Actual

Antes de ejecutar cualquier consulta parametrizada (**RUTA**, **SALTOS**, **ALCANZABLES** y **CERRAR**), el sistema verifica la validez de las estaciones en O(1) mediante **existeEstacion**(). Si una estación no pertenece a la red, imprime **“ERROR: estacion inexistente: X”** y continúa procesando el resto del archivo de consultas sin interrumpir el programa. Asimismo, maneja correctamente casos borde como rutas de una estación a sí misma (A = B), pares desconectados (NO HAY RUTA), y redes sin arcos (NO HAY RUTAS). El proyecto se encuentra 100% operativo y sin fallos conocidos.